

Fælles aner

Programmeringsprojekt

Elever: *Alexander Noah Morthorst, Louis Riis Nielsen, Tristan Noa Rosenberg*

Klasse: 3.O

Dato: *07/10/2026*

Indhold

1	Projektbeskrivelse	2
2	Funktionsbeskrivelse	3
3	Dokumentation af programmet	3
4	Kravedokumentation	5
5	Udviklingsprocessen	5
6	Bilag Kravedokumentation: produktet vs. kravene	5

1 Projektbeskrivelse

Synopsis til projektet *Fælles aner* (afsnit 7.10). Det fulde program ligger i repoet: https://github.com/ExperiencesXP/proj_heritage_tree på branchen `main`. I projektet modelleres en familie som en graf, hvor hver person er en knude, og hver forældreference (`mom/dad`) er en kant fra forælder til barn. Da en person højst har to forældre, er grafen tynd: der er aldrig flere end dobbelt så mange kanter som personer.

To idéer bærer projektet. For det første opdeles befolkningen i *klynger* — familier der hænger sammen: to personer hører til samme klynge, hvis man kan gå fra den ene til den anden ved at følge forældre- og børneforbindelser, uanset retning. For det andet findes en persons *forfædre* ved at følge mor- og far-referencerne opad, indtil der ikke er flere at følge — og hver person huskes undervejs, så genbesøg i sammenblandede familier kun behandles én gang. To personer er beslægtede, når deres forfædre overlapper.

Algoritmerne kan skrives som pseudo-kode:

```
klusterkort(personer):
    klynger = []
    besøgt = {}
    for hver person p, som ikke er i besøgt:
        klynge = {}                                # start en ny familie
        arbejdsliste = [p]
        mens arbejdsliste ikke er tom:
            q = tag_fra(arbejdsliste)
            hvis q er i besøgt: fortsæt
            læg q i besøgt og i klynge
            for hver nabo i [q.mor, q.far] + q.børn:
                hvis nabo findes og ikke er besøgt:
                    læg nabo i arbejdsliste
            læg klynge i klynger
    returnér klynger
```

```
forfædre(p):
    hvis p mangler: returnér tom mængde
    hvis p er i memo: returnér memo[p]          # hver person behandles
        kun én gang
    memo[p] = {p} + forfædre(p.mor) + forfædre(p.far)
    returnér memo[p]

fælles_ane(a, b):
    returnér sorteret liste over personer, som er
        i både forfædre(a) og forfædre(b)

er_i_familie(a, b):
    returnér True, hvis forfædre(a) og forfædre(b)
        har mindst ét fælles element
```

Fordi hver person og hver reference kun behandles et konstant antal gange, vokser køretiden lineært med familiens størrelse — målt til under en tiendedel sekund for 20.000 personer. Når slægtninge får børn sammen, optræder de samme forfædre flere gange i stamtræet; fordi programmet tæller hver person én gang, forbliver antallet af forfædre lille og håndterligt, selv i store familier. Antallet af klynger kan også aflæses direkte:

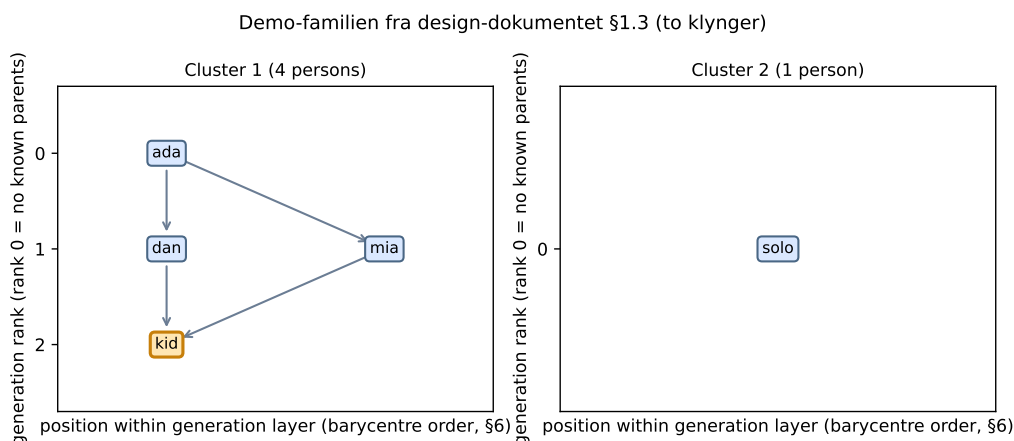
hver gang to dele af befolkningen samles, forsvinder én klynge, så klynger = personer – sammenlægninger.

2 Funktionsbeskrivelse

Brugeren installerer med `poetry install` og kører hele programmet med én kommando, `poetry run python src`. Programmet indlæser en befolkning af personer, bygger klusterkortet og svarer på spørgsmål om familien: hvilket kluster en person hører til, hvem personens forfædre og efterkommere er, en målrettet søgning med tidligt stop, og — som det centrale — `find_common_ancestor()` og `is_related()`, der besvarer spørgsmålet om fælles aner. Hvert kluster tegnes som et lagdelt stamtræ med `matplotlib.pyplot` og skrives til `out/`, og hvert kluster kan desuden eksporteres som Mermaid- eller Graphviz-kode.

Eksempel på udskrift:

```
1 common ancestors of mia and dan: ada
2 is_related(mia, dan) = True
3 is_related(kid, solo) = False
```

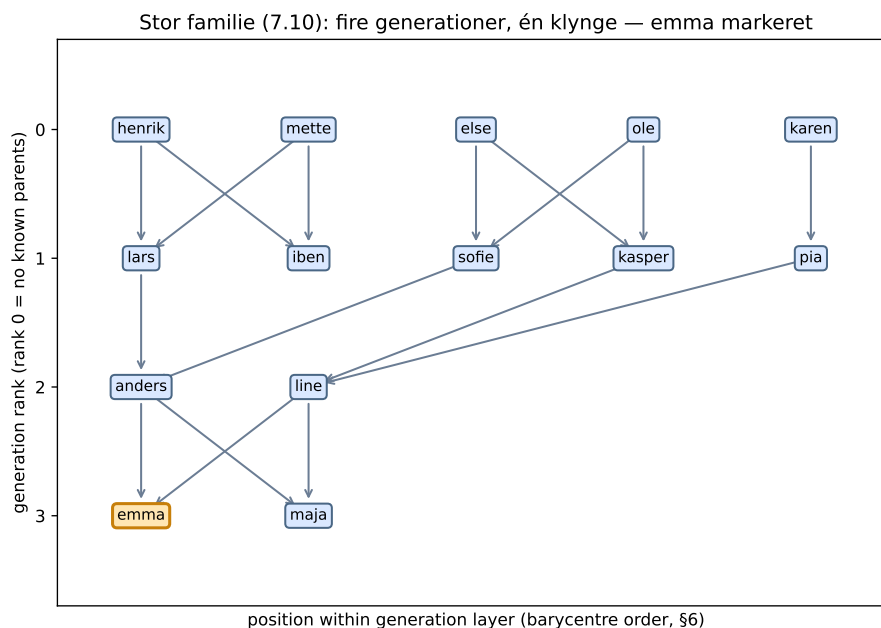


Figur 1: Demo-familien tegnet af programmets eget view-modul: to klynger — sammenbrudsfamilien til venstre og den isolerede person `solo` til højre.

Den indbyggede demo-familie har to klynger (figuren ovenfor): en sammenbrudt familie, hvor `ada` er forælder til både `mia` og `dan`, og den isolerede person `solo`. Opgavens *store familie* — fjorten personer over fire generationer — er tegnet herunder: et færdigt stamtræ, hvor programmets eget spørgsmål finder de fælles aner for de to forældre nederst, som er i familie med hinanden.

3 Dokumentation af programmet

Overordnet følger programmet MVC-arkitekturen, som opgaven kræver. *Modellen* er pakken `src/models/` med data og algoritmer og kender hverken til visning eller styring; *view*-laget (`draw.py`, `view.py`) omsætter resultater til tekstuddrag og matplotlib-figurer; og *controlleren* (`controller.py`, `__main__.py`) ejer tilstanden og kæder lagene sammen. Pipelinen er: indlæs befolkning → byg indeks → klusterkort → søgning → layout → eksport.



Figur 2: Opgavens store familie: fjorten personer, fire generationer, én klynge. Forældrene til **emma** og **maja** deler de samme to forfædre, hvilket programmet finder automatisk.

Programmets kerne er klasserne **Person** og **Pronouns** (datamodel), **FamilyGraph** (grafen), **Cluster**, **ClusterMap** og **UnionFind** (klusterkortet), **Layout** (tegningen) og **FamilyController** (styringen). Tabellen viser programmets vigtigste klasser og funktioner:

Modul	Klasser	Nøglefunktioner
models.person	Person, Pronouns	__str__, pronoun, from_preset
models.graph_model	FamilyGraph	build_family_graph, parent_edges
models.search	CycleError	explore, ancestors_iterative, find_common_ancestor, is_related
models.cluster_map	Cluster, UnionFind	build_cluster_map
draw	Layout	layout_cluster_map, to_mermaid, to_dot
view	—	render_cluster, render_cluster_map
controller	FamilyController	build_map, layout, is_related, render

OOP-principperne i koden er klasser og instanser, indkapsling (data og metoder hører sammen; controlleren gemmer sessionens tilstand), specialmetoden `__str__` der gør objekter læsbare ved udskrift, klassevariabler (`Pronouns.PRESETS`, `Person.DEFAULT_NAMES`), immutabilitet hvor det giver mening (`Pronouns` er en frossen dataklasse) og *composition* frem for nedarvning — familien bygges af objekter, der refererer hinanden, ikke af en ned-arvningshierarki. Standardnavnet på en person følger stedordene: **John Doe** ved han-ord, **Jane Doe** ved hun-ord, **Alex Doe** ved de-ord og **An Other** ellers.

Algoritmerne er valgt, så alle dele vokser lineært med familiens størrelse. Klusterkortet gennemgår hver person og hver reference én gang; klyngerne kan desuden samles løbende med en *union-find*-struktur, hvor hver person peger mod sin repræsentant, og stier halveres ved opslag, så sammenlægninger tager næsten konstant tid. Forfædre-udregningen

memoiserer: hver person gemmes efter første besøg, så store familier med sammenbandede slægtninge ikke regnes om. Søgninger bruger en eksPLICIT arbejdsliste frem for rekursion — det beskytter mod meget dybe familier — og en målrettet søgning stopper så snart svaret er fundet.

4 Kravdokumentation

Produktet lever op til kravene i Projekt.docx og afsnit 7.10; den punktvis dokumentation findes i bilaget bagerst. Kort sagt: programmet er skrevet i Python, visualiseringen bruger `matplotlib.pyplot` (view-laget), arkitekturen er MVC, og alle opgavens funktionskrav er opfyldt — inklusive `find_common_ancestor()` og `is_related()`, der besvarer spørgsmålet om fælles aner, og den store familie med stamtræ. To bevidste valg er dokumenteret i bilaget: koden er skrevet på engelsk, så attributterne `name/mom/dad` svarer til klassediagrammets `navn/mor/far`, og *fælles ane* tæller personen selv med, så en forælder og et barn regnes som beslægtede — ellers ville spørgsmålet give forkert svar for netop den relation.

5 Udviklingsprocessen

Arbejdet er styret af Git på `main`-branchen i det fælles repo. Først blev designet fastlagt: datamodel, algoritmer og hvilke tests der skulle vise, at løsningen virker. Derefter blev programmet implementeret i trin (datamodel → graf → søgning → klusterkort → layout → view), og tests er skrevet undervejs. Testmængden på 25 sammenligner klusterkortet med en uafhængig bredde-først-reference, tjekker på tilfældige familier, at antallet af klynger passer med antal personer minus sammenlægninger, og afprøver dybe kæder, cyklusser, eksporternes pilretning og slægtskabsspørgsmålet. Et review af koden fandt fire fejl, som alle blev rettet med regressionstests: en omvendt tekst i `__str__`, en vendt pilretning i de eksporterede diagrammer, en falsk cyklusfejl i målsøgningen og en rekursionsgrænse ved meget dybe familier. Udviklingen kan følges commit for commit med datoer i Git-historikken, og skærmdump heraf indsættes som dokumentation. **Arbejdsfordeling:** *(gruppen beskriver fordelingen her).*

Overvejelser omkring det færdige produkt: styrkerne er de enkle, gennemtænkte algoritmer og testmængden, som dokumenterer både korrekthed og hastighed, samt at programmet er lille og gennemskueligt. Begrænsningerne er, at befolkningen er indbygget i koden (ingen indlæsning fra fil eller database), at modellen følger opgaven og kun tillader to forældre pr. person, og at Mermaid/Graphviz-uddragene skal renderes eksternt. Naturlige udvidelser vil være filindlæsning, en grafisk brugerflade og en mere generel forældremodel.

6 Bilag Kravdokumentation: produktet vs. kravene

Kravene stammer fra Projekt.docx (produkt, visualisering, MVC) og afsnit 7.10 i bogen (*Projekt: Fælles aner*, s. 162–163). Koden er bevidst skrevet på engelsk; attributterne `name`, `mom`, `dad` svarer til diagrammets `navn`, `mor`, `far`.

Krav	Opfyldt af
Program der kan undersøge slægtskab (“Har to personer mindst en fælles ane?”)	<code>search.is_related()</code> og <code>find_common_ancestor()</code>
Person med valgfri mor og far	<code>Person(name, mom, dad)</code>
Klassediagrammets attributter	<code>name/mom/dad</code> (engelsk) + <code>pronouns</code>
Opret personer og forbind dem	<code>build_family_graph</code>
<code>__str__</code> à la “Navn Nikolaj, mors navn er Anne...”	<code>Person.__str__</code>
Test af <code>__str__</code>	<code>test_person_dunder_str_matches_book_style</code>
Standardnavne efter stedord (John/Jane/An Other)	<code>test_default_name_follows_pronouns</code>
Stor familie, mindst tre generationer	figur <code>stor_familie.pdf</code> (14 personer, 4 generationer)
Tegn et stamtræ	<code>view.render_cluster</code>
<code>find_common_ancestor()</code> returnerer liste over fælles aner	<code>search.find_common_ancestor()</code>
<code>is_related()</code> returnerer boolean	<code>search.is_related()</code>
Visualisering med <code>matplotlib.pyplot</code>	<code>src/view.py</code>
MVC-arkitektur	<code>src/models/</code> (model), <code>src/view.py</code> (view), <code>src/controller.py</code> (controller)

Svar på spørgsmålet “Hvilken type relation er der tale om?” Forbindelsen mellem `Person`-objekterne er en *association* — mere præcist *aggregering* (“har en”-relation): et objekt har referencer til andre `Person`-objekter som mor/far, objekterne eksisterer uafhængigt af hinanden, relationen er valgfri (`None`), og den er *refleksiv* — samme klasse i begge ender. Det er ikke nedarvning.

Definition-valg. Spørgsmålet er om *mindst* en fælles ane. Forfædre-mængden tæller også personen selv med, så en forælder og et barn tælles som beslægtede — ellers ville `is_related(ada, kid)` give `False`, uagtet at `ada` er `kid`s bedstemor.